

UE25CS251A
DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

ARCHITECTURE-Machine Language

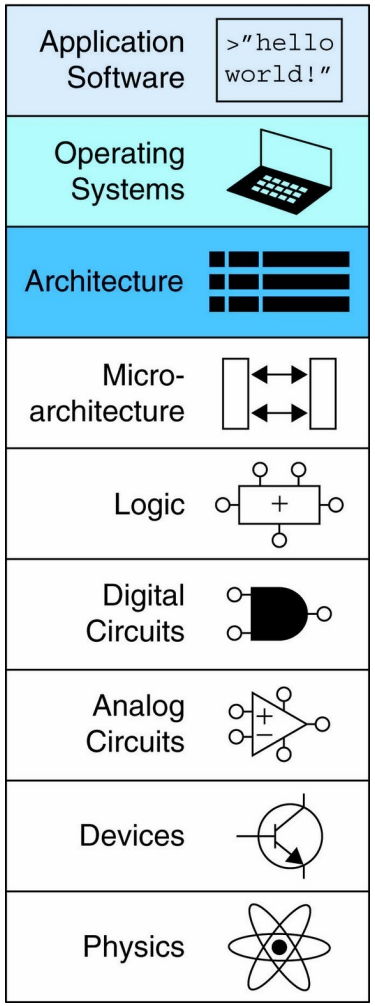
TEAM DLCD
Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

TOPICS

- Machine Language

R, I & J-Type Instructions
Instruction Formats
Logical Instructions
Shift Instructions
Stored Program
Machine Code Interpretation
Programming



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Machine Language

- Binary representation of instructions
- Computers only understand 1's and 0's
- 32–bit instructions
  - Simplicity favors regularity: 32–bit data & instructions
- 3 instruction formats:
  - **R – Type:** register operands
  - **I – Type:** immediate operands
  - **J – Type:** for jumping (discuss later)

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Machine Language: Big Picture

- A MIPS instruction is a 32-bit binary pattern stored in memory.
- The processor does not see add or lw; it only sees fields inside those 32 bits.
- Always read the opcode first, then decide whether the instruction is R-type, I-type, or J-type.
- The remaining fields tell the CPU which registers, constant, or address to use.
- TA shortcut: format first, meaning next.



Example: 001000 … → opcode = addi → interpret remaining bits as rs, rt, imm c (I-type) instruction

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Instruction Formats at a Glance

Format	Used for	Important fields	Quick memory rule
R-type	Register-register ALU ops	rs, rt, rd, funct	No immediate; funct decides exact op
I-type	Immediate, load/store, branch	rs, rt, imm	16-bit immediate/offset is present
J-type	Jump instructions	addr	26-bit jump target

Fast decoding order: opcode → format → field meaning → final assembly instruction

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

R-Type: Decode in 4 Steps

- R-type is used when the operands are mainly registers.
- If opcode = 000000, the instruction is R-type.
- rs and rt are usually the source registers; rd is the destination.
- funct chooses the exact ALU action such as add, sub, and, or, or shift.



Example: add $s0, $s1, $s2
rs = $s1, rt = $s2, rd = $s0, funct = 100000

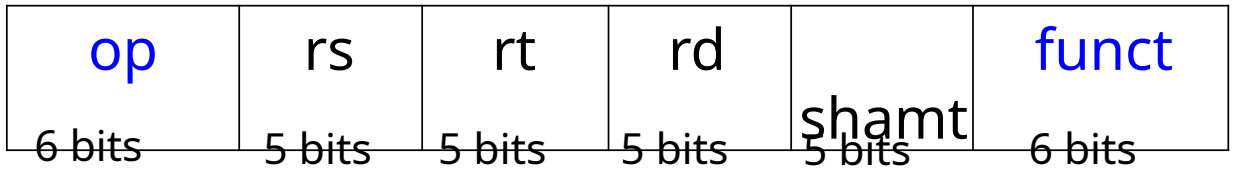
Memory tip: destination comes first in assembly, but is stored in rd.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

R – Type

- Register – type
- 3 register operands
  - rs, rt: source registers
  - rd: destination register
- Other fields:
  - op: the operation code or the opcode (0 for R–type instructions)
  - funct: the function with opcode tells computer what operation to perform
  - shamt: the shift amount for shift instructions, otherwise 0.
  - **Eg:For example, the opcode and funct fields for the add instruction are 0 (000000_2) and 32 (100000_2), respectively. Similarly, the sub instruction has an opcode and funct field of 0 and 34.**

R–Type



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

R – Type Examples

The machine code for the R-type instructions add and sub.
Notice that the destination is the first register in an assembly language instruction, but it is the third register field (rd) in the machine language instruction.
For example, the assembly instruction add $s0, $s1, $s2 has
rs = $s1 (17),
rt = $s2 (18),
rd = $s0 (16).

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
`add $s0, $s1, $s2`	0	17	18	16	0	32	000000	10001	10010	10000	00000	100000	(0x02328020)
`sub $t0, $t3, $t5`	0	11	13	8	0	34	000000	01011	01101	01000	00000	100010	(0x016D4022)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

R – Type Examples

Example 6.3 TRANSLATING ASSEMBLY LANGUAGE TO MACHINE LANGUAGE
Translate the following assembly language statement into machine language. \
add $t0, $s4, $s5

Solution: According to Table 6.1, $t0, $s4, and $s5 are registers 8, 20, and 21.
According to Tables B.1 and B.2, add has an opcode of 0 and a funct code of 32.

The easiest way to write the machine language in hexadecimal is to first write it in binary, then look at consecutive groups of four bits, which correspond to hexadecimal digits (indicated in blue). Hence, the machine language instruction is 0x02954020.

Table 6.1 MIPS register set

Name	Number	Use
$0	0	the constant value 0
$at	1	assembler temporary
$v0–$v1	2–3	function return value
$a0–$a3	4–7	function arguments
$t0–$t7	8–15	temporary variables
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	temporary variables
$k0–$k1	26–27	operating system (OS) temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

R – Type Examples

Example 6.3 TRANSLATING ASSEMBLY LANGUAGE TO MACHINE LANGUAGE
Translate the following assembly language statement into machine language.
add $t0, $s4, $s5

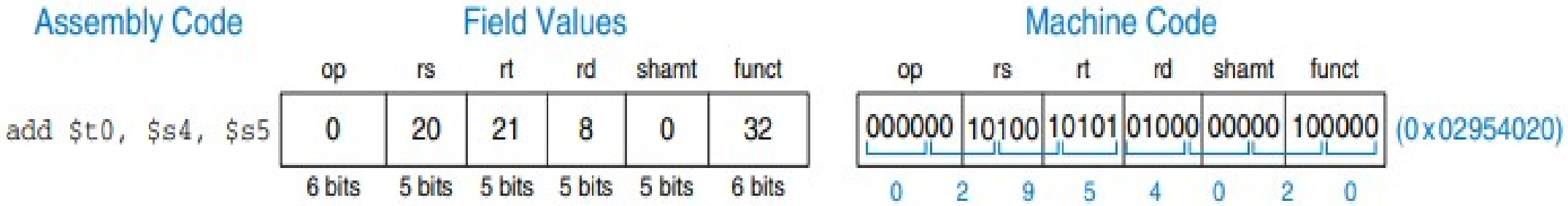


Figure 6.7 Machine code for the R-type instruction of Example 6.3

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

I – Type

- Immediate – type
- 2 register & 1 immediate (constant) operands
  - rs, rt: register operands
  - imm: 16–bit immediate for +ve number and 2's complement for –ve number
- Other fields:
  - op: the operation code or the opcode
  - Simplicity favors regularity: all instructions have opcode.
  - Operation is completely determined by opcode.

rs and imm are always used as source operands.
rt is used as a destination for some instructions (such as addi and lw) **but as another source for others (such as sw).**

I-Type



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

I – Type Examples

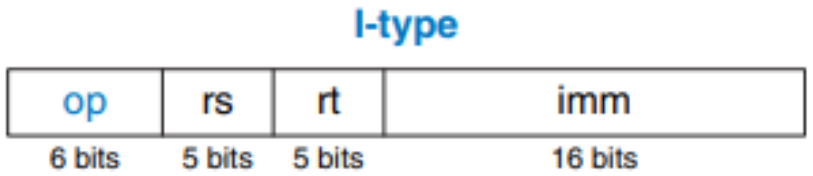


Figure 6.8 I-type instruction format

Assembly Code	op	rs	rt	imm	op	rs	rt	imm	Machine Code
addi $s0, $s1, 5	8	17	16	5	001000	10001	10000	0000 0000 0000 0101	(0x22300005)
addi $t0, $s3, -12	8	19	8	–12	001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
lw $t2, 32($0)	35	0	10	32	100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
sw $s1, 4($t1)	43	9	17	4	101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

Figure 6.9 Machine code for I-type instructions

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

I – Type Examples

Translate the following I-type instruction into machine code.
lw $s3, −24($s4)

Solution: According to Table 6.1, $s3 and $s4 are registers 19 and 20, respectively. **Tab**
B.1 indicates that lw has an opcode of 35.
rs specifies the base address, $s4,
and rt specifies the destination register, $s3.
The immediate, imm, encodes the 16-bit offset, −24. The fields and machine code are given in Figure 6.10.

Table 6.1 MIPS register set

Name	Number	Use
$0	0	the constant value 0
$at	1	assembler temporary
$v0–$v1	2–3	function return value
$a0–$a3	4–7	function arguments
$t0–$t7	8–15	temporary variables
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	temporary variables
$k0–$k1	26–27	operating system (OS) temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address



Figure 6.10 Machine code for an I-type instruction

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

I – Type Examples

NOTE: I-type instructions have a 16-bit immediate field, but the immediates are used in 32-bit operations. For example, lw adds a 16-bit offset to a 32-bit base register.

What should go in the upper half of the 32 bits?

For positive immediates, the upper half should be all 0's,
but for negative immediates, the upper half should be all 1's.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

I – Type Examples

- The addi instruction is helpful for assigning 16–bit constants.
- To assign 32–bit constants, use a load upper immediate instruction (lui) followed by an or immediate (ori) instruction as shown in code.

High-Level Code

```
int a = 0x6d5e4f3c;
```

MIPS Assembly Code

```
# $s0 = a
  lui $s0, 0x6d5e          # a = 0x6d5e0000
  ori $s0, $s0, 0x4f3c     # a = 0x6d5e4f3c
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

I-Type: Immediate, Load/Store, and Branch

- Format: op | rs | rt | imm
- imm is a 16-bit constant or address offset.
- For addi and lw, rt is usually the destination register.
- For sw, rt is the value that will be stored to memory.
- addi/lw/sw use sign extension; andi/ori/xori use zero extension.

Instruction	Read it as
addi $s0, $s1, 5	$s0 = $s1 + 5
lw $t2, 32($0)	$t2 = Memory[$0 + 32]
sw $s1, 4($t1)	Memory[$t1 + 4] = $s1

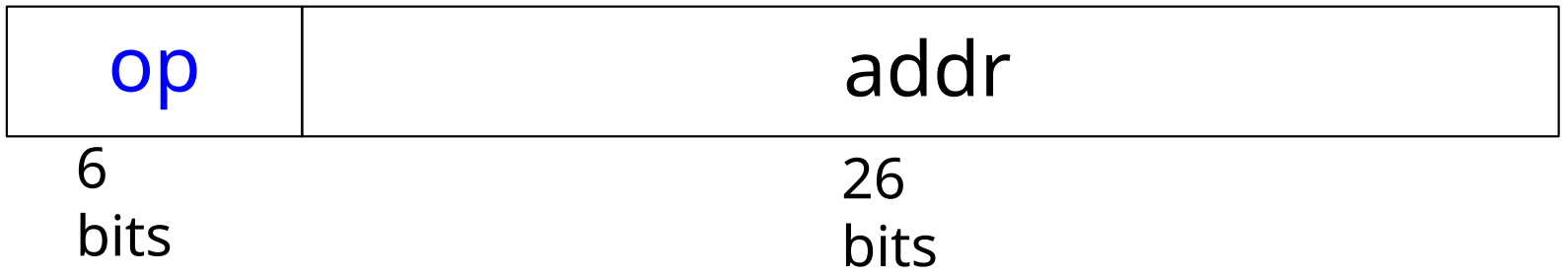
Exam check: in load/store, rs = base register, imm = offset.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

J – Type

- Jump – type
- 26–bit address operand (addr)
- Used for jump instructions (j)

J -Type

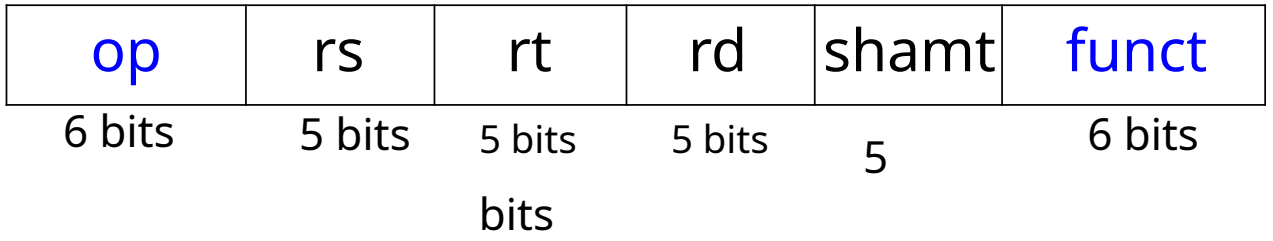


© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

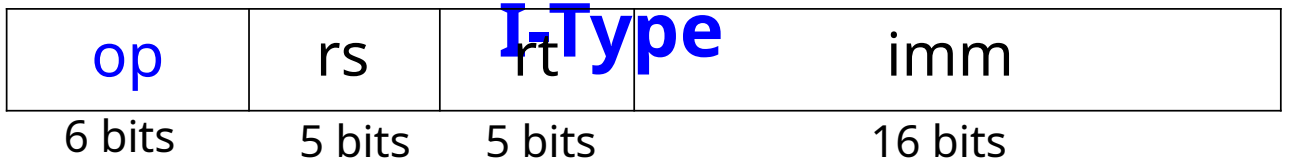
DIGITAL LOGIC AND COMPUTER DESIGN

Review: Instruction Formats

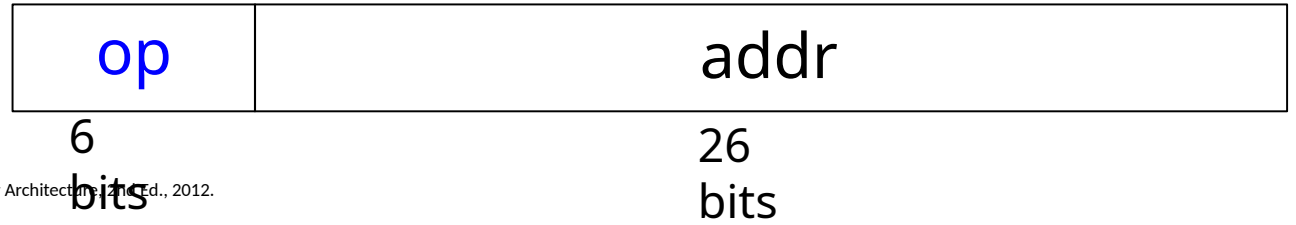
R-Type



I-Type



J -Type



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Interpreting Machine Code

- To interpret machine language, one must decipher the fields of each 32-bit instruction word.
- Different instructions use different formats, but all formats start with a 6-bit opcode field.
- Thus, the best place to begin is to look at the opcode. If it is 0, the instruction is R-type.
- otherwise it is I-type or J-type.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Interpreting Machine Code

Example 6.5 TRANSLATING MACHINE LANGUAGE TO ASSEMBLY LANGUAGE

Translate the following machine language code into assembly language.

```
0x2237FFF1
0x02F34022
```

Solution: First, we represent each instruction in binary and look at the six most significant bits to find the `opcode` for each instruction, as shown in Figure 6.12. The `opcode` determines how to interpret the rest of the bits. The `opcodes` are 001000_2 (8_{10}) and 000000_2 (0_{10}), indicating an `addi` and R-type instruction, respectively. The `funct` field of the R-type instruction is 100010_2 (34_{10}), indicating that it is a `sub` instruction. Figure 6.12 shows the assembly code equivalent of the two machine instructions.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Machine Code Decoding Strategy

- Step 1: Read the opcode.
- Step 2: If opcode = 0, treat it as R-type and inspect funct.
- Step 3: Otherwise, use the opcode itself to identify the I-type or J-type instruction.
- Step 4: Convert register numbers carefully to register names.
- Step 5: Interpret the immediate as signed or unsigned based on the instruction.

Worked example: 001000 10001 10111 1111111111110001

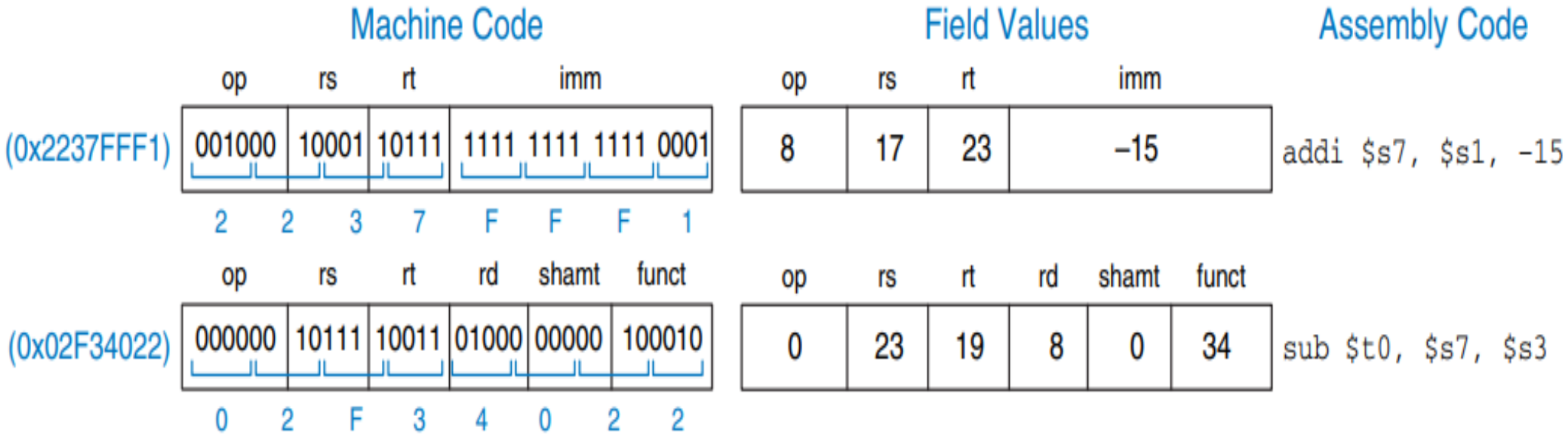
Field	Value	Meaning
opcode	001000	addi
rs	10001	$s1
rt	10111	$s7
imm	1111…0001	-15

Final answer: addi $s7, $s1, -15

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Interpreting Machine Code



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Review: The Stored Program Concept

What is a Stored Program?

- A program is stored in **main memory as machine code (binary instructions)**.
- Each instruction occupies a **unique memory address**.
- The processor executes instructions **one at a time** in sequence.

Machine Code:

- Executed directly by the CPU.
- Represented in **binary or hexadecimal** format.
- Difficult for humans to read and write.
- **Example:** 0x8C0A0020

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Review: The Stored Program Concept

Assembly Code:

- Human-readable representation of machine instructions.
- Uses **mnemonics** (e.g., lw, add, sub) instead of binary.
- Easier to write, understand, and debug.
- **Example:** lw $t2, 32($0)

Program Counter (PC):

- The **Program Counter (PC)** is a special register that stores the **address of the next instruction** to be executed.
- After an instruction is fetched, the PC is **normally incremented by 4 bytes** (for MIPS 32-bit instructions).
- Branch and jump instructions modify the PC to execute instructions from a different location.

Key point: A stored-program computer keeps both instructions and data in memory. The CPU uses the Program Counter (PC) to fetch and execute instructions sequentially unless a branch or jump changes the execution flow.

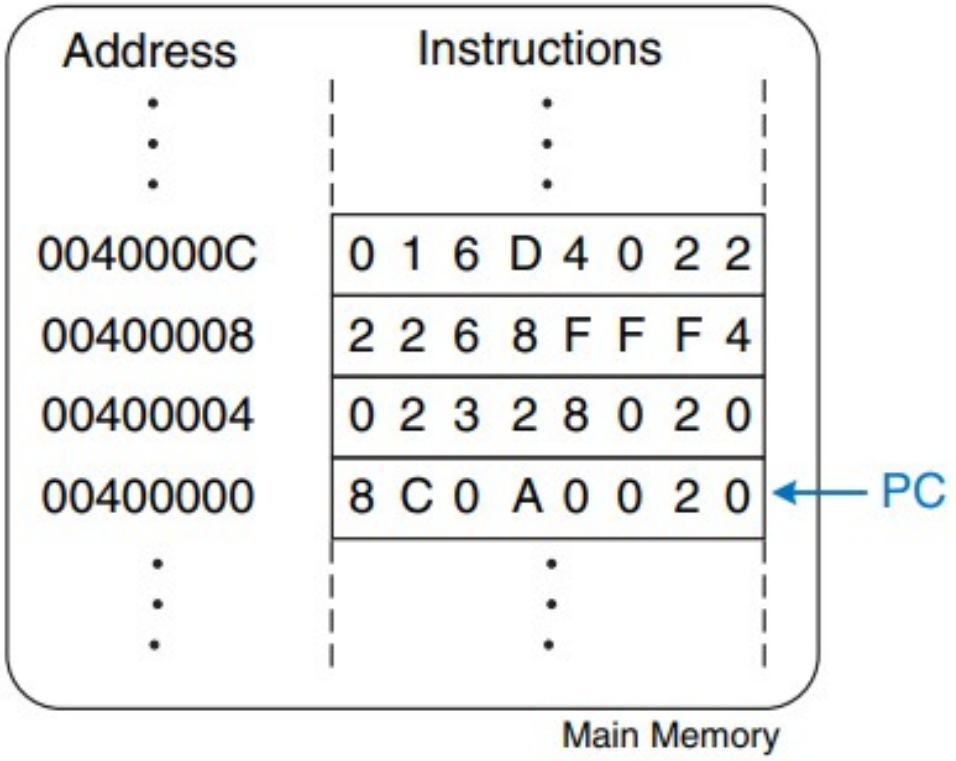
© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Review: The Stored Program Concept

Assembly Code	Machine Code
`lw $t2, 32($0)`	0x8C0A0020
`add $s0, $s1, $s2`	0x02328020
`addi $t0, $s3, -12`	0x2268FFF4
`sub $t0, $t3, $t5`	0x016D4022

Stored Program



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

- **Q1. An operand stored in memory is accessed using which instructions in MIPS?**
  A) add and sub
  B) lw and sw
  C) beq and j
  D) and and or

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

MCQ

- **Q2. In an R-type instruction, which field identifies the exact ALU operation when opcode = 0?**

  A) imm
  B) funct
  C) shamt
  D) addr

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

- **Q3. Which set of immediate instructions uses zero extension?**
  A) addi, lw, sw
  B) andi, ori, xori
  C) beq, bne
  D) j

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

- **Q4. In sra, the new most significant bits are filled using:**
  A) 0s
  B) sign bit
  C) funct bits
  D) shamt bits

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

- **Q5. In sllv rd, rt, rs, the shift amount comes from:**
  A) shamt field
  B) lower 5 bits of rs
  C) lower 5 bits of rt
  D) opcode

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

- **Q6. Which instruction format is used by j?**
  A) R-type
  B) I-type
  C) J-type
  D) depends on funct

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Answer Key

Q1: B Q2: B Q3: B Q4: B Q5: B Q6: C

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.



THANK YOU

Team DLCD

Department of CSE and CSE(AI & ML)